zk-SNARKs Membership Verification

9th Dec 2025

Jeong Yunseo

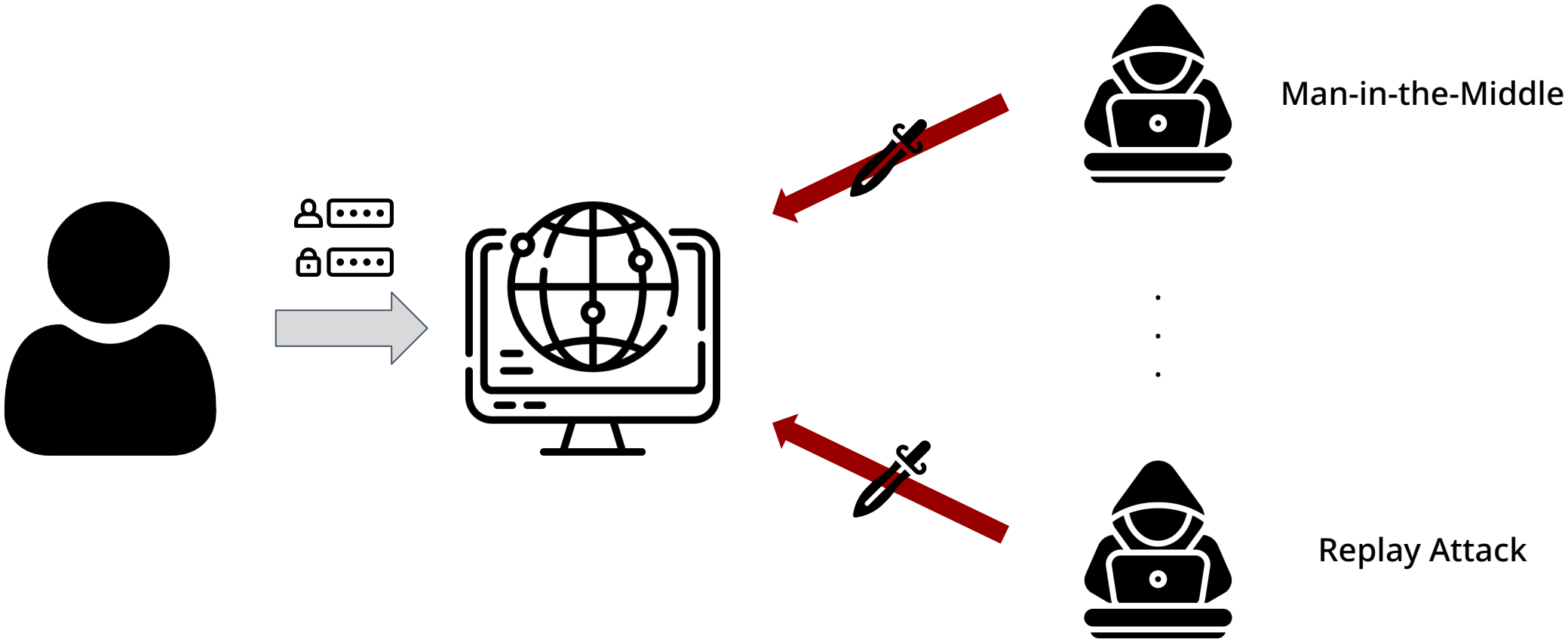
jeong.ys@unist.ac.kr

ULSAN NATIONAL INSTITUTE OF SCIENCE AND TECHNOLOGY

Contents

1. Introduction
2. Motivation
3. Proposed Idea
4. Evaluation
5. Limitation
6. Future Work
7. Conclusion

Introduction



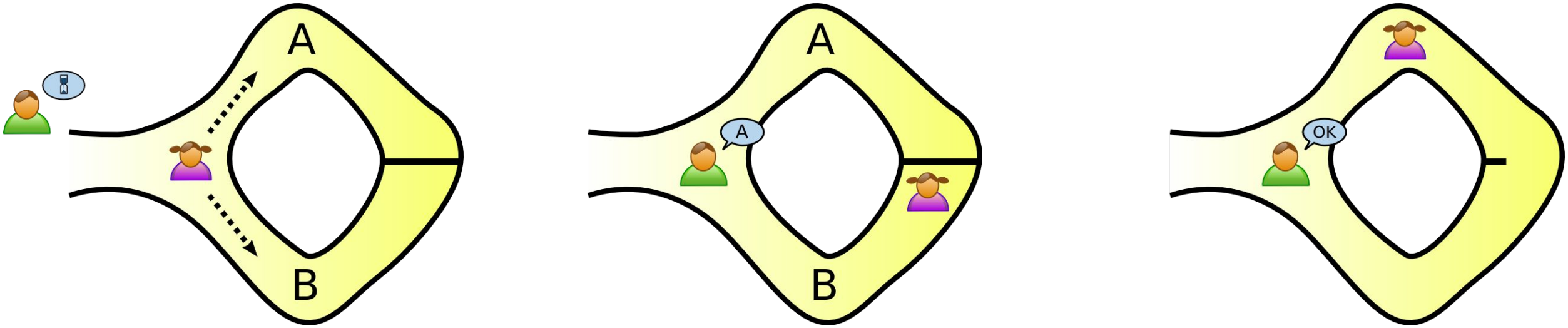
Introduction

Enable anonymous yet verifiable authentication in network handshakes

Background

Zero Knowledge Proof

: a protocol in which one party (the prover) can convince another party (the verifier) that some given statement is true, without conveying to the verifier any information beyond the mere fact of that statement's truth



https://en.wikipedia.org/wiki/Zero-knowledge_proof

Background

Zero Knowledge Proof (ZKP)

- Classical ZKP is interactive and produce large proofs.

zk-SNARKs

- solve these issues
- non-interactive, compact, and fast

1. **Fiat-Shamir** (⇒ non-interactive)

   : replaces the verifier's random challenge with a hash function
2. **Q**uadratic **A**rithmetic **P**rogram (⇒ compact)

   : compresses an entire computation into a single polynomial equation
3. **Bilinear Pairings** (⇒ fast)

   allow the verifier to check the polynomial relation using only a few elliptic-curve operations

Motivation

Online ZK Handshake

- Embeds session information (ctx, A, B) inside the circuit
- Strong protection against replay attacks
- Ensures the proof is valid only for a specific session

Motivation

Limitations of Online ZK Handshake

- Requires running fullProve() for every session
- Proving time is very high (hundreds to thousands of milliseconds)
- Results in large latency, unsuitable for real-world applications

I need a design that preserves the security of ZK, while also achieving real-time session authentication performance.

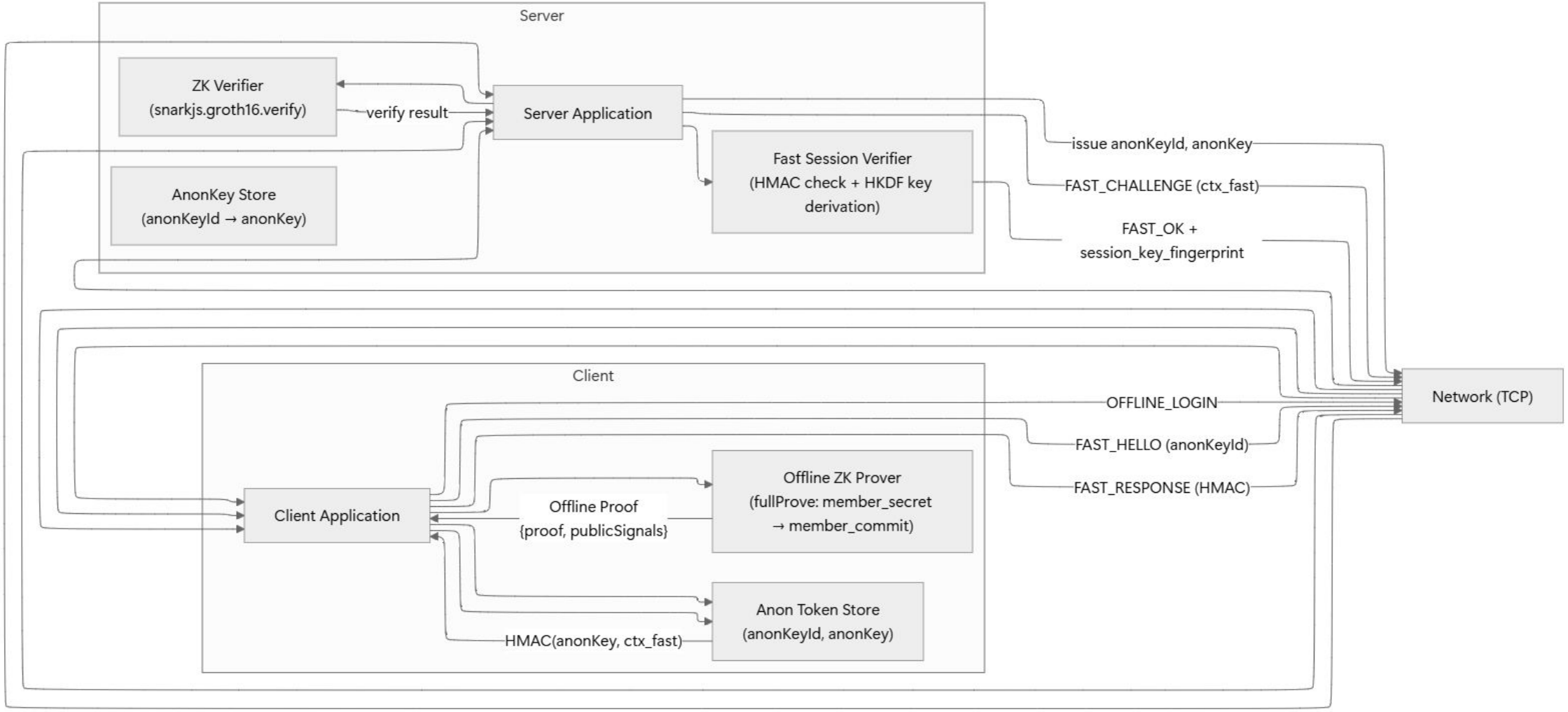
Motivation

Major Threats

- Replay Attacks
- Man-in-the-Middle (MITM) Attacks
- Proof / PublicSignals Tampering
- Token Theft (anonKey compromise)
- Session Key Manipulation Attacks

Proposed Idea

Architecture



Proposed Idea

Offline Membership Proof (ZK)

- ZK proof is generated only once
- Server verifies membership and issues an anonKey (ID-free token)
- User identity remains hidden, only membership is proven

Fast Session Binding (MAC-based)

- Client computes HMAC (anonKey, ctx_fast)
- Server validates MAC to authenticate the session
- Replay attacks impossible due to fresh ctx_fast
- Session authentication latency becomes dramatically lower

Proposed Idea

Core Components

- publicSignals: [policy_id, member_commit]
- private input: member_secret
- Circuit logic:

  commit = Poseidon(member_secret)

  → Prover proves ownership of the commitment without revealing the secret.

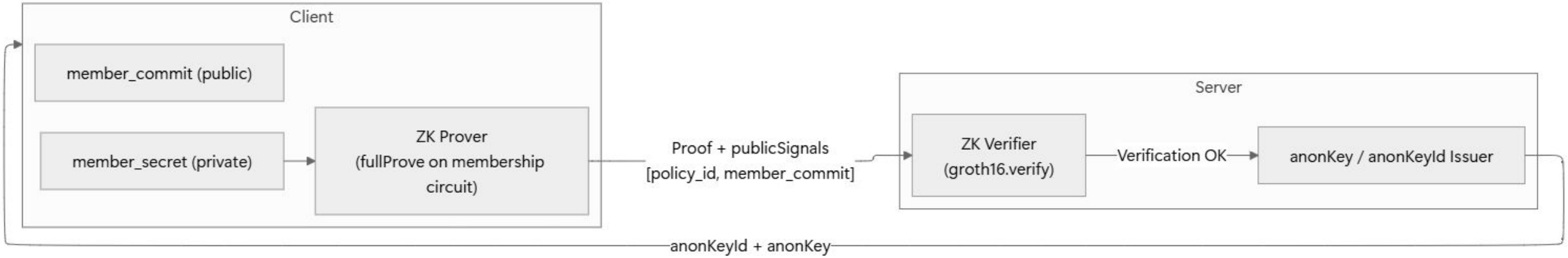
Proposed Idea

Server-Side Verification

- Server verifies the proof with groth16.verify()
- If verification succeeds:

  Server issues anonKeyId and anonKey

  Becomes the client's long-term anonymous



Proposed Idea

Fast Session Phase (MAC-based)

Protocol Flow

- Client → Server: FAST_HELLO (send anonKeyId)
- Server → Client: send fresh ctx_fast
- Client → Server: compute and send HMAC(anonKey, ctx_fast)
- Server: verifies the MAC
- On success:
  → derive session key: HKDF(anonKey || ctx_fast)

Key Advantages

- Tens to hundreds of times faster than ZK-based online proving
- Replay attacks are fully prevented
  - each session uses a fresh ctx_fast
  - attacker cannot forge MAC without anonKey

Proposed Idea

Threat Model

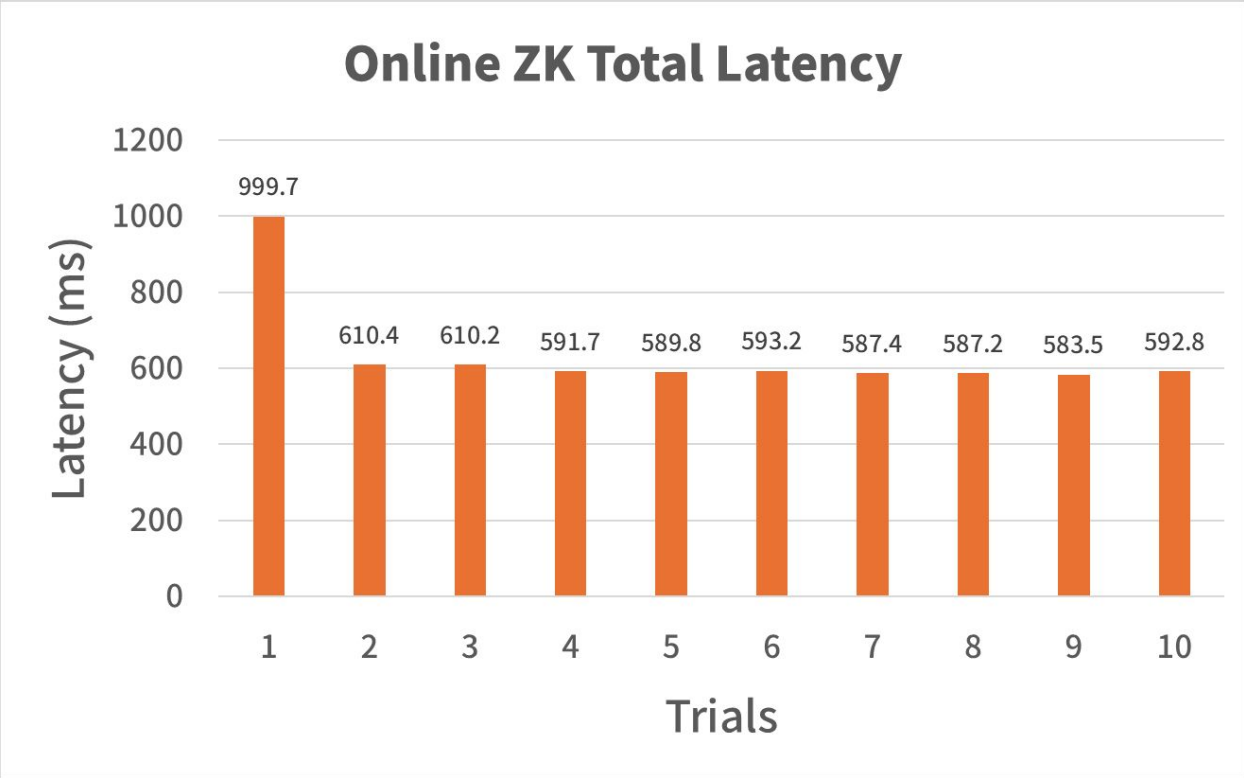
Key Attacks	Defenses
Replay Attacks	• Mitigated by fresh ctx_fast + HMAC (anonKey, ctx_fast)
Proof / PublicSignals Tampering	• Blocked by strict consistency checks on all public signals (A_field, B_field, ctx_field, ctx_hash)
Man-in-the-Middle Attacks	• Any modification leads to proof or ctx_hash mismatch • verification fails
Token Theft (anonKey compromise)	• Recognized as a storage-layer risk • requires secure client storage

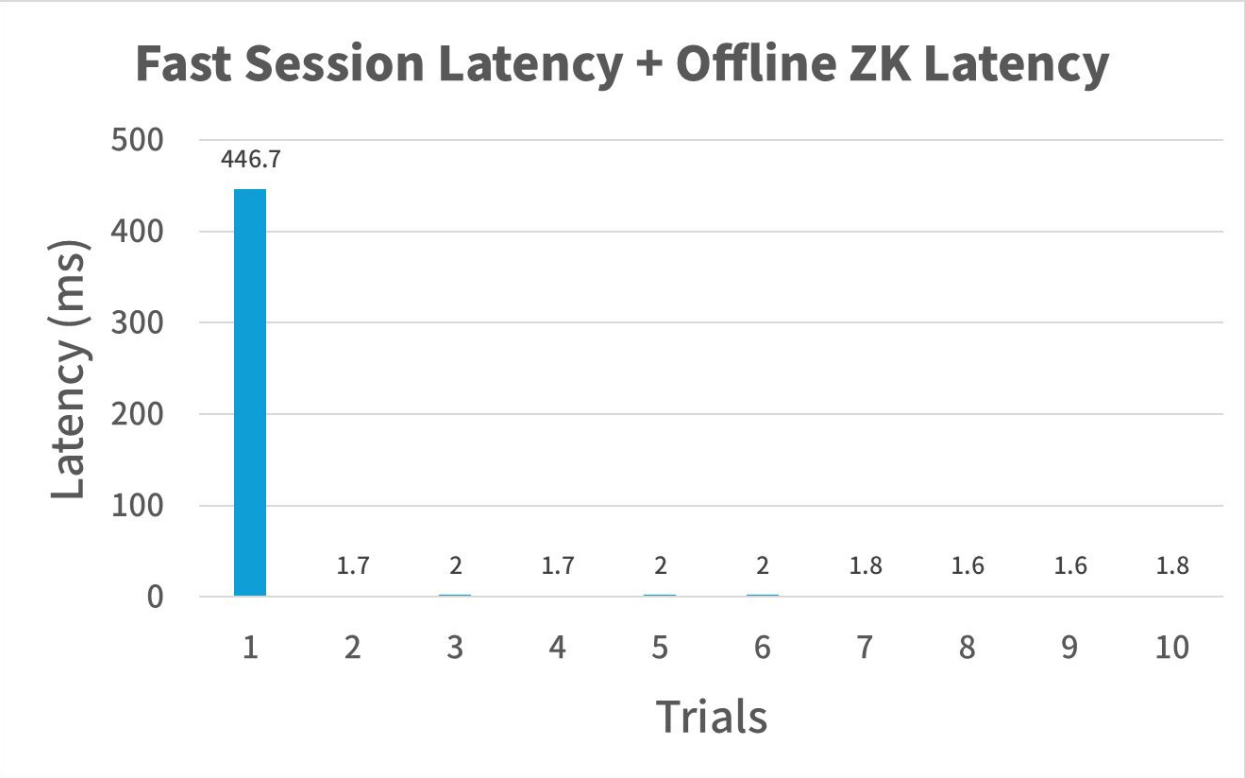
Evaluation

Latency Comparison

Fast session is about at least twice as fast.

The first trial includes one-time ZK proving overhead.





Evaluation

Replay Attack Resistance

```
n2sl@20255384-p02:~/zknet-handshake$ node offline_server.js
offline_server listening on 4000
Incoming: OFFLINE_LOGIN
verify: 378 ms
Offline login OK for policy 1 commit 12073538098082875406557387017103187984392382 0
3455489736088771687285407239702 3
Incoming: FAST_HELLO
Incoming: FAST_HELLO
```

```
n2sl@20255384-p02:~/zknet-handshake$ node offline_login_client.js
Generating OFFLINE proof...
fullProve: 429 ms
Proof generated. Connecting to server...
Server: {
  ok: true,
  type: 'OFFLINE_LOGIN_OK',
  anonKeyId: '0bec49ed1e2e2fffedb4b6af52a80f94',
  anonKey: '64834f8e92306544bde4f20187391277870578a9e9580d465c39314ca481c1d2'
}
anon_token.json saved
n2sl@20255384-p02:~/zknet-handshake$ node replay_offline.js
=== Phase 1: Normal Fast Session to capture MAC ===
[phase1] FAST_CHALLENGE received
[phase1] server response: {
  ok: true,
  type: 'FAST_OK',
  session_key_fingerprint: '0110e16f9be447e82aec1c4353989dfceb636c776e06a87123388d
49e95330c9'
}
Captured MAC from first session: 8507e3c66f26e4499440a052b18f979bec7fa66325d45120c
63f6b1f9adaa94d
Captured ctx_fast (base64): dTCPDGmSN1opva10dUSmyKJsSiDJhMZIuHstZ4U+0as=

=== Phase 2: Replay Attack with OLD MAC ===
[replay] connected, sending FAST_HELLO with anonKeyId
[replay] server sent NEW ctx_fast (IGNORED)
[replay] sending OLD MAC from previous session
[replay] server final response: { ok: false, reason: 'FAST_AUTH_FAIL' }
[replay] Replay blocked as expected. reason = FAST_AUTH_FAIL

Done.
```

Limitation

- anonKey compromise can lead to session impersonation

  → real deployments require secure storage (Secure Enclave / TPM) and TLS
- Offline proof is reusable

  → makes usage tracking impossible, which may not fit all policy models
- ZK proving is still computationally expensive

  → even if moved offline, large circuits incur noticeable cost
- Membership revocation is non-trivial

  → requires a clear anonKey invalidation / expiration policy

Future Work

Accelerating ZK Proving

- Explore GPU-based parallel proving to reduce proving latency
- Investigate DPU / SmartNIC acceleration for offloading hash-heavy components

Conclusion

- This work addresses the high latency of traditional online ZK handshakes by introducing an offline ZK + MAC-based fast session architecture
- The proposed system provides privacy preservation, strong replay resistance, and robust MITM protection
- Experimental results show that the Fast Session mechanism is tens to hundreds of times faster than Online ZK proving

ULSAN NATIONAL INSTITUTE OF SCIENCE AND TECHNOLOGY
2007

THANK YOU